SESSION 04 / PRINTABLE GUIDE

Lesson 4: Surface notebook-prepared data in Warehouse Gold

Session 04 | Shared publication walkthrough for both examples
Output: Physical Warehouse Gold tables populated from verified Lakehouse transfer tables.

Micro-fixture SQL: The executable contracts and counts here belong only to the four-order/nine-event examples. For realistic datasets, follow main-lab publication and its separate `s04_lab_` SQL assets. The handoff/safety principles below apply to both tiers; their table names, schemas, and expected results are not interchangeable.

What actually moves?

A notebook variable is not visible to Warehouse SQL. A Lakehouse Delta table does not automatically become a Warehouse table because both use OneLake.

The handoff is an explicit transfer of flat, typed records:

```text
Successful notebook + reviewed quality/count/batch evidence
  -> Lakehouse Delta transfer table
  -> Verified copy into Warehouse staging
  -> SQL validation and transactional Gold replacement
  -> Model freshness step and intended-consumer check
```

The Lakehouse SQL analytics endpoint is a query surface, not the place to issue T-SQL inserts into Lakehouse tables. Run the SQL below in the training **Warehouse**.

Before this handoff, use the column-selection walkthrough to explain why accepted Silver retains useful business/context/lineage fields while the transfer contract is narrower. Notebook exploration and Data Wrangler help discover transformations; neither publishes Warehouse Gold.

1. Prepare fixed contracts in an isolated Warehouse

Use one dedicated training Warehouse per pair and no concurrent writers. The table names are intentionally prefixed `s04_` to avoid Session 03 objects. Run setup once. If a target already exists with unknown data or an incompatible definition, stop and use a fresh isolated target; do not drop or repair it automatically.

```sql
IF SCHEMA_ID('stg') IS NULL EXEC('CREATE SCHEMA stg');
IF SCHEMA_ID('gold') IS NULL EXEC('CREATE SCHEMA gold');

CREATE TABLE stg.s04_accepted_order_line
(
    batch_id varchar(64) NOT NULL,
    order_id varchar(20) NOT NULL,
    line_number int NOT NULL,
    customer_id varchar(20) NOT NULL,
    customer_name varchar(100) NOT NULL,
    sku varchar(30) NOT NULL,
    quantity int NOT NULL,
    unit_price decimal(18,2) NOT NULL,
    line_amount decimal(18,2) NOT NULL
);

CREATE TABLE gold.s04_accepted_order_line
(
    batch_id varchar(64) NOT NULL,
    order_id varchar(20) NOT NULL,
    line_number int NOT NULL,
    customer_id varchar(20) NOT NULL,
    customer_name varchar(100) NOT NULL,
    sku varchar(30) NOT NULL,
    quantity int NOT NULL,
    unit_price decimal(18,2) NOT NULL,
    line_amount decimal(18,2) NOT NULL,
    published_at_utc datetime2(6) NOT NULL
);

CREATE TABLE stg.s04_device_daily
(
    batch_id varchar(64) NOT NULL,
    reading_date date NOT NULL,
    device_id varchar(20) NOT NULL,
    reading_count bigint NOT NULL,
    min_temperature_c decimal(8,2) NOT NULL,
    max_temperature_c decimal(8,2) NOT NULL,
    avg_temperature_c decimal(8,2) NOT NULL
);

CREATE TABLE gold.s04_device_daily
(
    batch_id varchar(64) NOT NULL,
    reading_date date NOT NULL,
    device_id varchar(20) NOT NULL,
    reading_count bigint NOT NULL,
    min_temperature_c decimal(8,2) NOT NULL,
    max_temperature_c decimal(8,2) NOT NULL,
```

```
    avg_temperature_c decimal(8,2) NOT NULL,
    published_at_utc datetime2(6) NOT NULL
);
```

The contracts set explicit lengths, numeric precision, and grain. The supplied fixtures fit these lengths. Real transformations must also validate string lengths, approved currency/units, schema versions, and all required business fields before transfer.

2. Default handoff: Copy Lakehouse tables into Warehouse staging

Configure two copy activities if demonstrating both examples:

Setting	Orders	Devices
Source item	Approved training Lakehouse	Approved training Lakehouse
Root	Tables	Tables
Actual source table	`s04_order_lines_transfer`	`s04_device_daily_transfer`
Warehouse destination	`stg.s04_accepted_order_line`	`stg.s04_device_daily`
Table option	Use existing	Use existing
Write behavior	Insert	Insert
Mapping	Explicitly map the nine transfer columns	Explicitly map the seven transfer columns
Expected copied rows	3	2, accounting for 4 accepted events

Select the actual schema-qualified Lakehouse table shown in the tenant if schemas are enabled. Do not copy nested Silver columns or raw JSON into the flat Warehouse contract.

Warehouse copy may need compatible temporary staging. Verify the supported staging option, storage/network access, and connection identity in the actual tenant. Workspace staging or an approved external staging connection are documented options, not a guarantee of Government availability. Do not invent a direct `COPY INTO` over an arbitrary OneLake URL.

For repeatable fixture loading, use the appropriate reviewed **pre-copy script** for that activity:

```sql
DELETE FROM stg.s04_accepted_order_line
WHERE batch_id = 's04-orders-001';
```

```sql
DELETE FROM stg.s04_device_daily
WHERE batch_id = 's04-devices-001';
```

These scripts touch only the named staging batch. Never configure a pre-copy DELETE/TRUNCATE against the live Gold table. A failed transfer can leave staging empty or partial while the previous Gold stays intact.

Serialize the notebook, copy, validation, and publication for each example. Disable automatic racing retries and do not start a second notebook overwrite while copy is reading its transfer table. Notebook completion alone does not provide a durable cross-service batch manifest.

Before publication: Require notebook success, immutable input/batch evidence, reviewed quarantine acceptance, expected transfer counts, successful copy, and a matching staging batch. The fixture checks below are additional guards, not replacements for those controls.

If the transfer path is blocked, stop here and label Warehouse publication unverified. A manually typed result or SQL-only seed would not prove the notebook-to-Warehouse handoff.

3. Publish accepted order lines

Run this as a separate Warehouse execution after the orders copy succeeds. It replaces the entire current teaching Gold snapshot, not an incremental subset of a production fact table.

```sql
DECLARE @BatchId varchar(64) = 's04-orders-001';

BEGIN TRY
    BEGIN TRANSACTION;

    IF (SELECT COUNT(*) FROM stg.s04_accepted_order_line
        WHERE batch_id = @BatchId) <> 3
        THROW 51001, 'Expected three staged accepted order lines.', 1;

    IF EXISTS
    (
        SELECT order_id, line_number
        FROM stg.s04_accepted_order_line
        WHERE batch_id = @BatchId
```

```
        GROUP BY order_id, line_number
        HAVING COUNT(*) > 1
    )
        THROW 51002, 'Staged order-line keys must be unique.', 1;

    IF EXISTS
    (
        SELECT 1 FROM stg.s04_accepted_order_line
        WHERE batch_id = @BatchId
          AND (TRIM(order_id) = '' OR TRIM(customer_id) = ''
               OR TRIM(customer_name) = '' OR TRIM(sku) = ''
               OR line_number <= 0 OR quantity <= 0 OR unit_price < 0
               OR line_amount <> CAST(quantity * unit_price AS decimal(18,2)))
    )
        THROW 51003, 'Staged accepted-line contract failed.', 1;

    IF (SELECT SUM(line_amount) FROM stg.s04_accepted_order_line
        WHERE batch_id = @BatchId) <> 296.00
        THROW 51004, 'Unexpected fixture accepted-line total.', 1;

    DELETE FROM gold.s04_accepted_order_line;

    INSERT INTO gold.s04_accepted_order_line
    (
        batch_id, order_id, line_number, customer_id, customer_name,
        sku, quantity, unit_price, line_amount, published_at_utc
    )
    SELECT
        batch_id, order_id, line_number, customer_id, customer_name,
        sku, quantity, unit_price, line_amount, SYSUTCDATETIME()
    FROM stg.s04_accepted_order_line
    WHERE batch_id = @BatchId;

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH;

SELECT order_id, line_number, sku, quantity, unit_price, line_amount
FROM gold.s04_accepted_order_line
ORDER BY order_id, line_number;
```

Compare all three business rows with the orders example, not just the total. Label 296.00 as an accepted-line amount. Rejected orders/lines remain in Lakehouse quarantine for investigation.

4. Publish device/day summaries

Run separately after the devices copy succeeds.

```sql
DECLARE @BatchId varchar(64) = 's04-devices-001';

BEGIN TRY
    BEGIN TRANSACTION;

    IF (SELECT COUNT(*) FROM stg.s04_device_daily
        WHERE batch_id = @BatchId) <> 2
        THROW 51101, 'Expected two staged device/day rows.', 1;

    IF EXISTS
    (
        SELECT device_id, reading_date
        FROM stg.s04_device_daily
        WHERE batch_id = @BatchId
        GROUP BY device_id, reading_date
        HAVING COUNT(*) > 1
    )
        THROW 51102, 'Staged device/day keys must be unique.', 1;

    IF EXISTS
    (
        SELECT 1 FROM stg.s04_device_daily
        WHERE batch_id = @BatchId
          AND (TRIM(device_id) = '' OR reading_count <= 0
               OR reading_date <> CAST('2026-10-28' AS date)
               OR min_temperature_c < -40 OR max_temperature_c > 85
               OR min_temperature_c > max_temperature_c
               OR avg_temperature_c < min_temperature_c
               OR avg_temperature_c > max_temperature_c)
    )
        THROW 51103, 'Staged device summary contract failed.', 1;

    IF (SELECT SUM(reading_count) FROM stg.s04_device_daily
        WHERE batch_id = @BatchId) <> 4
        THROW 51104, 'Expected four accepted device events.', 1;

    DELETE FROM gold.s04_device_daily;

    INSERT INTO gold.s04_device_daily
    (
        batch_id, reading_date, device_id, reading_count,
        min_temperature_c, max_temperature_c, avg_temperature_c,
        published_at_utc
    )
    SELECT
        batch_id, reading_date, device_id, reading_count,
        min_temperature_c, max_temperature_c, avg_temperature_c,
        SYSUTCDATETIME()
    FROM stg.s04_device_daily
```

```
    WHERE batch_id = @BatchId;

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH;

SELECT reading_date, device_id, reading_count,
       min_temperature_c, max_temperature_c, avg_temperature_c
FROM gold.s04_device_daily
ORDER BY reading_date, device_id;
```

Verify D01 has two accepted readings and 20.00/24.00/22.00 minimum/maximum/average; D02 has two and 30.00/32.00/31.00. These fixture-specific guards are intentionally not a generic production load framework.

The two Gold publications are independent transactions. This walkthrough does not provide atomic publication across both examples.

5. Optional handoff: Spark connector writes to Warehouse staging

Microsoft documents `DataFrame.write.synapsesql` for Warehouse writes. The documentation reviewed for this lesson specifies Runtime 2.0 and **interactive Entra user authentication only**, with service principal authentication unsupported. Do not turn a successful interactive demonstration into a claim that the intended scheduled identity is supported.

Use this route only after checking the actual runtime, tenant/cloud availability, Warehouse permissions, network/staging requirements, and execution identity. No `pip install`, hard-coded token, or ad hoc JDBC credential is required by the documented integrated connector.

After clearing only the intended staging batch with the reviewed SQL above, an orders example is:

PYTHON

```python
import com.microsoft.spark.fabric
from com.microsoft.spark.fabric.Constants import Constants

transfer = spark.table("s04_order_lines_transfer")
transfer.write.mode("append").synapsesql(
    "TrainingWarehouse.stg.s04_accepted_order_line"
)
```

Replace `TrainingWarehouse` with the actual approved Warehouse name. This is a write to **Warehouse staging**, not to the Lakehouse SQL analytics endpoint and not directly to Gold.

Append is safe here only with the explicit staging cleanup, isolated target, one writer, completed batch evidence, and subsequent SQL validation. A blind retry can duplicate staging rows and must not be described as idempotent. Verify type mapping against the existing table.

Use either copy activity or the connector for a given transfer, not both. The SQL publication and consumer checks remain the same.

6. Prove the consumer and the failure boundary

Create/inspect a shared semantic model over the physical Gold tables using a verified mode. For orders, report accepted-line count and accepted-line amount. For devices, report accepted-reading count and the published device/day metrics.

Do not average daily averages across days/devices without an approved weighting rule. Where the intended measure is an average over accepted events, retain/derive an appropriate sum and count rather than treating every daily average as equally weighted.

Verify the mode-specific model freshness behavior, the Gold batch/publication metadata, known values, and the intended reader's access. SQL/OneLake permissions and semantic-model rules are separate boundaries.

Demonstrate an unchanged rerun and confirm identical business rows, allowing publication timestamps to change. Discuss a failed-copy or missing-batch scenario: do not run publication, preserve the last valid Gold, and record the incomplete current run. If testing an actual rejection, use only a reviewed isolated fixture/staging change and restore it afterward; do not disrupt a shared connection.

Sources and Government verification

These are product references reviewed on October 28, 2026, not proof of GCC High availability:

- Load Lakehouse data with a notebook
- Lakehouse copy activity configuration
- Warehouse copy activity configuration
- Spark connector for Fabric Data Warehouse

Use paths/endpoints copied from the target environment, not hard-coded commercial-cloud hostnames. Live Spark, connector/copy, SQL, scheduling, and identity execution remain unverified by this source package.

Source: 04_warehouse_gold.md. Companion-material links open the repository originals.